

LAB 2a

BACKEND VỚI NODE.JS — MICROSERVICES

RESTful API • Swagger • Prisma • PostgreSQL • MongoDB • Docker • Deploy

I. MỤC TIÊU BÀI HỌC

Sau khi hoàn thành Lab này, sinh viên có khả năng:

- Thiết kế và xây dựng hệ thống Microservices với Node.js + Express theo chuẩn công nghiệp.
- Kết nối và thao tác với PostgreSQL thông qua Prisma ORM (schema-first, type-safe).
- Kết nối và thao tác với MongoDB thông qua Mongoose ODM.
- Tích hợp Swagger UI để tự động sinh tài liệu API (OpenAPI 3.0).
- Xây dựng API đầy đủ: validation, error handling chuẩn, pagination, filtering, sorting.
- Container hoá ứng dụng bằng Docker & Docker Compose.
- Deploy lên môi trường free host: Railway / Render / Supabase / MongoDB Atlas.

II. KIẾN TRÚC MICROSERVICES

2.1 Tổng quan kiến trúc

Thay vì xây dựng một ứng dụng monolithic, chúng ta sẽ tách thành các service độc lập, mỗi service có database riêng và giao tiếp qua HTTP (REST) hoặc Message Queue.

Service	Port	Database	Chức năng
API Gateway	3000	— (proxy)	Route request tới các service
Product Service	3001	PostgreSQL (Prisma)	Quản lý sản phẩm, danh mục
Order Service	3002	MongoDB (Mongoose)	Quản lý đơn hàng, giỏ hàng
Auth Service	3003	PostgreSQL (Prisma)	Xác thực JWT, phân quyền

2.2 Cấu trúc thư mục tổng thể

```
microservices-shop/  
├── api-gateway/      # Điểm vào duy nhất của hệ thống  
└── src/
```



III. CHUẨN BỊ MÔI TRƯỜNG

3.1 Phần mềm cần cài đặt

Công cụ	Phiên bản	Tải tại
Node.js	≥ 18.x LTS	nodejs.org
Docker Desktop	≥ 24.x	docker.com
VS Code	Latest	code.visualstudio.com
Postman / Thunder Client	Latest	postman.com
DBeaver (tùy chọn)	Latest	dbeaver.io — quản lý DB GUI

3.2 Tài khoản cần tạo

- MongoDB Atlas — mongodb.com/atlas (Free M0 Cluster)
- Supabase — supabase.com (PostgreSQL Free tier)
- Railway — railway.app (Deploy backend free \$5 credit)
- GitHub — để lưu code và kết nối với nền tảng deploy

3.3 VS Code Extensions được khuyến nghị

- Prisma (Prisma.prisma) — syntax highlighting cho schema.prisma
- REST Client — test API ngay trong VS Code
- Docker (ms-azuretools.vscode-docker)
- ESLint + Prettier — format code tự động
- GitLens — xem lịch sử git chi tiết

IV. XÂY DỰNG PRODUCT SERVICE (PostgreSQL + Prisma)

4.1 Khởi tạo Project

```
mkdir product-service && cd product-service
npm init -y

# Cài đặt dependencies chính
npm install express prisma @prisma/client dotenv cors
npm install express-validator swagger-jsdoc swagger-ui-express
npm install helmet morgan compression

# Dev dependencies
npm install --save-dev nodemon @types/node

# Khởi tạo Prisma
npx prisma init --datasource-provider postgresql
```

4.2 Biến môi trường (.env)

```
# .env - KHÔNG commit file này lên Git!
NODE_ENV=development
PORT=3001
SERVICE_NAME=product-service

# PostgreSQL - lấy từ Supabase dashboard
DATABASE_URL="postgresql://postgres:[PASSWORD]@db.[PROJECT].supabase.co:5432/postgres"

# Dùng connection pooling cho production
DATABASE_URL_POOL="postgresql://postgres.[PROJECT]:[PASSWORD]@aws-0-ap-southeast-1.pooler.supabase.com:6543/postgres?pgbouncer=true"
```

 **Ghi chú** Tạo file .env.example với giá trị placeholder (không có password thật) và commit file này lên Git để người khác biết cần cấu hình gì.

4.3 Prisma Schema — Định nghĩa cấu trúc Database

Mở file `prisma/schema.prisma` và định nghĩa các bảng:

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Bảng danh mục sản phẩm
model Category {
  id          Int          @id @default(autoincrement())
  name        String       @unique
  slug        String       @unique
  description  String?
  createdAt   DateTime      @default(now()) @map("created_at")
  updatedAt   DateTime      @updatedAt      @map("updated_at")
  products    Product[]

  @@map("categories")
}

// Bảng sản phẩm chính
model Product {
  id          Int          @id @default(autoincrement())
  name        String
  slug        String       @unique
  description  String?
  price       Decimal      @db.Decimal(10, 2)
  stock       Int          @default(0)
  imageUrl    String?      @map("image_url")
  isActive    Boolean      @default(true) @map("is_active")
  categoryId   Int?         @map("category_id")
  category    Category?    @relation(fields: [categoryId], references: [id])
  createdAt   DateTime      @default(now()) @map("created_at")
  updatedAt   DateTime      @updatedAt      @map("updated_at")

  @@index([categoryId])
  @@index([isActive])
  @@map("products")
}
```

Chạy migration để tạo bảng trong database:

```
# Tạo migration và áp dụng vào DB
npx prisma migrate dev --name init

# Xem database trong Prisma Studio (GUI)
npx prisma studio

# Generate Prisma Client sau mỗi lần sửa schema
npx prisma generate
```

💡 **Ghi chú** Prisma tự động tạo TypeScript types từ schema. Bạn sẽ có autocomplete đầy đủ khi code.

4.4 Seed dữ liệu mẫu

```
// prisma/seed.js
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

async function main() {
  // Tạo categories
  const mobile = await prisma.category.upsert({
    where: { slug: "mobile" },
    update: {},
    create: { name: "Điện thoại", slug: "mobile", description: "Điện thoại smartphone" }
  });

  // Tạo products
  await prisma.product.createMany({
    data: [
      { name: "iPhone 15 Pro", slug: "iphone-15-pro", price: 27990000, stock: 50, categoryId: mobile.id },
      { name: "Samsung Galaxy S24", slug: "samsung-s24", price: 22990000, stock: 30, categoryId: mobile.id },
    ],
    skipDuplicates: true
  });
  console.log("✅ Seed thành công!");
}

main().catch(console.error).finally(() => prisma.$disconnect());
```

Thêm vào package.json:

```
"prisma": {
  "seed": "node prisma/seed.js"
}

# Chạy seed
npx prisma db seed
```

4.5 Controller với Prisma Client

```
// src/controllers/productController.js
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

// -----
// GET /api/products - Lấy danh sách có phân trang, lọc, sắp xếp
// -----
const getProducts = async (req, res, next) => {
  try {
    const {
      page = 1, limit = 10,
      search = "", category,
      sortBy = "createdAt", order = "desc",
      minPrice, maxPrice, inStock
    } = req.query;
```

```

    } = req.query;

    const skip = (parseInt(page) - 1) * parseInt(limit);

    // Xây dựng điều kiện filter
    const where = {
      isActive: true,
      ...(search && { name: { contains: search, mode: "insensitive" } }),
      ...(category && { category: { slug: category } }),
      ...(minPrice || maxPrice) && {
        price: {
          ...(minPrice && { gte: parseFloat(minPrice) }),
          ...(maxPrice && { lte: parseFloat(maxPrice) }),
        }
      },
      ...(inStock === "true" && { stock: { gt: 0 } }),
    };

    // Chạy song song 2 queries
    const [products, total] = await Promise.all([
      prisma.product.findMany({
        where,
        include: { category: { select: { name: true, slug: true } } },
        orderBy: { [sortBy]: order },
        skip,
        take: parseInt(limit),
      }),
      prisma.product.count({ where }),
    ]);

    res.json({
      success: true,
      data: products,
      pagination: {
        total, page: parseInt(page), limit: parseInt(limit),
        totalPages: Math.ceil(total / parseInt(limit))
      }
    });
  } catch (error) {
    next(error); // Chuyển lỗi sang errorHandler
  }
};

// _____
// GET /api/products/:id
// _____
const getProductById = async (req, res, next) => {
  try {
    const product = await prisma.product.findUnique({
      where: { id: parseInt(req.params.id) },
      include: { category: true },
    });
    if (!product) return res.status(404).json({ success: false, message: "Không tìm thấy sản phẩm" });
    res.json({ success: true, data: product });
  } catch (error) { next(error); }
};

// _____
// POST /api/products
// _____
const createProduct = async (req, res, next) => {
  try {

```

```

    const { name, price, description, stock, imageUrl, categoryId } = req.body;
    const slug = name.toLowerCase().replace(/\s+/g, "-").replace(/^[^a-z0-9-]/g, "");

    const product = await prisma.product.create({
      data: { name, slug, price, description, stock, imageUrl, categoryId },
      include: { category: true }
    });
    res.status(201).json({ success: true, data: product, message: "Tạo sản phẩm thành công" });
  } catch (error) { next(error); }
};

// =====
// PUT /api/products/:id
// =====
const updateProduct = async (req, res, next) => {
  try {
    const product = await prisma.product.update({
      where: { id: parseInt(req.params.id) },
      data: req.body,
      include: { category: true }
    });
    res.json({ success: true, data: product, message: "Cập nhật thành công" });
  } catch (error) { next(error); }
};

// =====
// DELETE /api/products/:id (Soft delete)
// =====
const deleteProduct = async (req, res, next) => {
  try {
    await prisma.product.update({
      where: { id: parseInt(req.params.id) },
      data: { isActive: false } // Soft delete - không xoá thật
    });
    res.json({ success: true, message: "Đã ẩn sản phẩm thành công" });
  } catch (error) { next(error); }
};

module.exports = { getProducts, getProductById, createProduct, updateProduct, deleteProduct };

```

4.6 Middleware: Validation & Error Handling

```

// src/middleware/validate.js - Dùng express-validator
const { body, validationResult } = require("express-validator");

const handleValidation = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(422).json({
      success: false,
      message: "Dữ liệu không hợp lệ",
      errors: errors.array().map(e => ({ field: e.path, message: e.msg }))
    });
  }
  next();
};

const productValidation = [
  body("name").notEmpty().withMessage("Tên sản phẩm không được rỗng")

```

```

        .isLength({ min: 2, max: 200 }).withMessage("Tên từ 2-200 ký tự"),
        body("price").isFloat({ min: 0 }).withMessage("Giá phải là số dương"),
        body("stock").optional().isInt({ min: 0 }).withMessage("Số lượng phải là số nguyên ≥ 0"),
        handleValidation,
    ];

module.exports = { productValidation };

// _____
// src/middleware/errorHandler.js
// _____
const errorHandler = (err, req, res, next) => {
    console.error(`[${new Date().toISOString()}] ERROR:`, err);

    // Prisma unique constraint violation
    if (err.code === "P2002") {
        return res.status(409).json({ success: false, message: `${err.meta?.target} đã tồn tại` });
    }
    // Prisma record not found
    if (err.code === "P2025") {
        return res.status(404).json({ success: false, message: "Không tìm thấy bản ghi" });
    }

    res.status(err.status || 500).json({
        success: false,
        message: err.message || "Lỗi hệ thống",
        ...(process.env.NODE_ENV === "development" && { stack: err.stack })
    });
};

module.exports = errorHandler;

```

V. TÍCH HỢP SWAGGER (OpenAPI 3.0)

Swagger giúp tự động tạo tài liệu API dạng giao diện web, đồng thời cho phép test API trực tiếp trên trình duyệt.

5.1 Cài đặt

```
npm install swagger-jsdoc swagger-ui-express
```

5.2 Cấu hình Swagger

```

// src/swagger/swagger.js
const swaggerJsdoc = require("swagger-jsdoc");

const options = {
    definition: {
        openapi: "3.0.0",
        info: {
            title: "Product Service API",

```

```

    version: "1.0.0",
    description: "API quản lý sản phẩm – Lab 2 Nâng cao",
    contact: { name: "Dev Team", email: "dev@example.com" }
  },
  servers: [
    { url: "http://localhost:3001", description: "Development" },
    { url: "https://product-service.railway.app", description: "Production" }
  ],
  components: {
    schemas: {
      Product: {
        type: "object",
        properties: {
          id: { type: "integer", example: 1 },
          name: { type: "string", example: "iPhone 15 Pro" },
          price: { type: "number", example: 27990000 },
          stock: { type: "integer", example: 50 },
          description: { type: "string", example: "Mô tả sản phẩm..." },
          imageUrl: { type: "string", example: "https://..." },
          isActive: { type: "boolean", example: true },
          category: { $ref: "#/components/schemas/Category" }
        }
      },
      Category: {
        type: "object",
        properties: {
          id: { type: "integer", example: 1 },
          name: { type: "string", example: "Điện thoại" },
          slug: { type: "string", example: "mobile" }
        }
      },
      PaginatedProducts: {
        type: "object",
        properties: {
          success: { type: "boolean" },
          data: { type: "array", items: { $ref: "#/components/schemas/Product" } },
          pagination: {
            type: "object",
            properties: {
              total: { type: "integer" },
              page: { type: "integer" },
              limit: { type: "integer" },
              totalPages: { type: "integer" }
            }
          }
        }
      }
    }
  },
  // Tự động scan JSDoc comments từ các file route
  apis: ["./src/routes/*.js"],
};

module.exports = swaggerJsdoc(options);

```

5.3 Viết JSDoc annotations cho Routes

```

// src/routes/productRoutes.js
const router = require("express").Router();

```

```

const { getProducts, getProductById, createProduct, updateProduct, deleteProduct }
    = require("../controllers/productController");
const { productValidation } = require("../middleware/validate");

/**
 * @swagger
 * /api/products:
 *   get:
 *     summary: Lấy danh sách sản phẩm
 *     tags: [Products]
 *     parameters:
 *       - in: query
 *         name: page
 *         schema: { type: integer, default: 1 }
 *         description: Số trang
 *       - in: query
 *         name: limit
 *         schema: { type: integer, default: 10 }
 *         description: Số bản ghi mỗi trang
 *       - in: query
 *         name: search
 *         schema: { type: string }
 *         description: Tìm kiếm theo tên
 *       - in: query
 *         name: category
 *         schema: { type: string }
 *         description: Lọc theo slug danh mục
 *       - in: query
 *         name: minPrice
 *         schema: { type: number }
 *       - in: query
 *         name: maxPrice
 *         schema: { type: number }
 *       - in: query
 *         name: sortBy
 *         schema: { type: string, enum: [name, price, createdAt], default:
createdAt }
 *       - in: query
 *         name: order
 *         schema: { type: string, enum: [asc, desc], default: desc }
 *     responses:
 *       200:
 *         description: Thành công
 *         content:
 *           application/json:
 *             schema:
 *               $ref: "#/components/schemas/PaginatedProducts"
 */
router.get("/", getProducts);

/**
 * @swagger
 * /api/products:
 *   post:
 *     summary: Tạo sản phẩm mới
 *     tags: [Products]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [name, price]

```

```

*           properties:
*             name:      { type: string }
*             price:     { type: number }
*             description: { type: string }
*             stock:     { type: integer }
*             categoryId: { type: integer }
*       responses:
*         201:
*           description: Tạo thành công
*         422:
*           description: Dữ liệu không hợp lệ
*/
router.post("/", productValidation, createProduct);

router.get("/:id", getProductById);
router.put("/:id", updateProduct);
router.delete("/:id", deleteProduct);

module.exports = router;

```

5.4 Tích hợp vào app.js

```

// src/app.js
const express = require("express");
const cors    = require("cors");
const helmet   = require("helmet");
const morgan   = require("morgan");
const swaggerUi = require("swagger-ui-express");
const swaggerSpec = require("../swagger/swagger");
const productRoutes = require("../routes/productRoutes");
const errorHandler = require("../middleware/errorHandler");
require("dotenv").config();

const app = express();

// — Security & Logging Middleware —
app.use(helmet()); // Bảo mật HTTP headers
app.use(cors());
app.use(morgan("dev")); // Log requests
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// — Swagger UI —
app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerSpec, {
  swaggerOptions: { persistAuthorization: true },
  customSiteTitle: "Product Service API Docs",
}));

// Export spec dạng JSON để tích hợp với API Gateway
app.get("/api-docs.json", (req, res) => res.json(swaggerSpec));

// — Routes —
app.get("/health", (req, res) => res.json({
  status: "ok", service: process.env.SERVICE_NAME, uptime: process.uptime()
}));
app.use("/api/products", productRoutes);

// — Global Error Handler (ĐẶT CUỐI CÙNG) —
app.use(errorHandler);

module.exports = app;

```

💡 **Ghi chú** Truy cập <http://localhost:3001/api-docs> để xem Swagger UI với đầy đủ tài liệu và khả năng test API tương tác.

VI. XÂY DỰNG ORDER SERVICE (MongoDB + Mongoose)

Order Service quản lý đơn hàng. MongoDB phù hợp cho dữ liệu đơn hàng vì cấu trúc linh hoạt — một đơn hàng có thể chứa nhiều items với các thuộc tính khác nhau.

6.1 Khởi tạo Project

```
mkdir order-service && cd order-service
npm init -y
npm install express mongoose dotenv cors helmet morgan
npm install express-validator swagger-jsdoc swagger-ui-express
npm install --save-dev nodemon
```

6.2 Biến môi trường (.env)

```
NODE_ENV=development
PORT=3002
SERVICE_NAME=order-service

# MongoDB Atlas — lấy từ Connect > Drivers
MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.xxx.mongodb.net/ordersDB?retryWrites=true&w=majority

# URL của Product Service (để validate sản phẩm)
PRODUCT_SERVICE_URL=http://localhost:3001
```

6.3 Mongoose Models

```
// src/models/Order.js
const mongoose = require("mongoose");

// Sub-schema cho mỗi sản phẩm trong đơn hàng
const OrderItemSchema = new mongoose.Schema({
  productId: { type: Number, required: true }, // ID từ Product Service
  productName: { type: String, required: true }, // Snapshot tại thời điểm mua
  price: { type: Number, required: true }, // Giá tại thời điểm mua
  quantity: { type: Number, required: true, min: 1 },
  subtotal: { type: Number, required: true },
}, { _id: false });

const OrderSchema = new mongoose.Schema({
  orderCode: { type: String, unique: true }, // Tự sinh: ORD-20240115-001
  customerId: { type: Number, required: true }, // ID từ Auth Service
  customerName: { type: String, required: true },
  customerEmail: { type: String, required: true },
  items: { type: [OrderItemSchema], required: true },
  totalAmount: { type: Number, required: true },
  status: {
```

```

    type: String,
    enum: ["pending", "confirmed", "shipping", "delivered", "cancelled"],
    default: "pending"
  },
  shippingAddress: {
    street: String,
    city: String,
    district: String,
  },
  note: String,
}, {
  timestamps: true, // Tự thêm createdAt & updatedAt
  versionKey: false, // Bỏ __v field
});

// Middleware: tự sinh orderCode trước khi save
OrderSchema.pre("save", async function(next) {
  if (!this.orderCode) {
    const date = new Date().toISOString().slice(0,10).replace(/-/g, "");
    const count = await mongoose.model("Order").countDocuments();
    this.orderCode = `ORD-${date}-${String(count + 1).padStart(4, "0")}`;
  }
  next();
});

// Virtual: tính tổng số lượng items
OrderSchema.virtual("totalItems").get(function() {
  return this.items.reduce((sum, item) => sum + item.quantity, 0);
});

// Index để tìm kiếm nhanh
OrderSchema.index({ customerId: 1, createdAt: -1 });
OrderSchema.index({ status: 1 });

module.exports = mongoose.model("Order", OrderSchema);

```

6.4 Controller đơn hàng

```

// src/controllers/orderController.js
const Order = require("../models/Order");

// Tạo đơn hàng mới
const createOrder = async (req, res, next) => {
  try {
    const { customerId, customerName, customerEmail, items, shippingAddress, note } =
      req.body;

    // Tính tổng tiền & subtotal từng item
    const processedItems = items.map(item => ({
      ...item,
      subtotal: item.price * item.quantity,
    }));
    const totalAmount = processedItems.reduce((sum, i) => sum + i.subtotal, 0);

    const order = await Order.create({
      customerId, customerName, customerEmail,
      items: processedItems, totalAmount,
      shippingAddress, note
    });

    res.status(201).json({ success: true, data: order });
  }

```

```

    } catch (error) { next(error); }
  };

// Lấy đơn hàng của customer, có phân trang
const getOrdersByCustomer = async (req, res, next) => {
  try {
    const { customerId } = req.params;
    const { page = 1, limit = 10, status } = req.query;
    const skip = (parseInt(page) - 1) * parseInt(limit);

    const filter = { customerId: parseInt(customerId) };
    if (status) filter.status = status;

    const [orders, total] = await Promise.all([
      Order.find(filter).sort({ createdAt: -1 }).skip(skip).limit(parseInt(limit)),
      Order.countDocuments(filter)
    ]);

    res.json({ success: true, data: orders,
      pagination: { total, page: parseInt(page), totalPages: Math.ceil(total /
limit) }
    });
  } catch (error) { next(error); }
};

// Cập nhật trạng thái đơn hàng
const updateOrderStatus = async (req, res, next) => {
  try {
    const { status } = req.body;
    const order = await Order.findByIdAndUpdate(
      req.params.id,
      { status },
      { new: true, runValidators: true }
    );
    if (!order) return res.status(404).json({ success: false, message: "Không tìm thấy
đơn hàng" });
    res.json({ success: true, data: order });
  } catch (error) { next(error); }
};

module.exports = { createOrder, getOrdersByCustomer, updateOrderStatus };

```

VII. CONTAINER HOÁ VỚI DOCKER

7.1 Dockerfile cho mỗi Service

```

# Dockerfile (dùng chung cho product-service và order-service)
# Dùng multi-stage build để giảm kích thước image

# Stage 1: Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Chỉ áp dụng cho product-service (có Prisma)
COPY prisma ./prisma
RUN npx prisma generate

```

```
# Stage 2: Production
FROM node:18-alpine AS production
WORKDIR /app
ENV NODE_ENV=production

COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/prisma ./prisma
COPY src ./src
COPY package.json ./

EXPOSE 3001
CMD ["node", "src/index.js"]
```

7.2 Docker Compose — Chạy toàn bộ hệ thống

```
# docker-compose.yml
version: "3.9"

networks:
  microservices-net:
    driver: bridge

volumes:
  postgres_data:
  mongo_data:

services:
  # — Databases —
  postgres:
    image: postgres:15-alpine
    container_name: postgres_db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: products_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    networks: [microservices-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin"]
      interval: 10s
      timeout: 5s
      retries: 5

  mongo:
    image: mongo:7
    container_name: mongo_db
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: secret
      MONGO_INITDB_DATABASE: orders_db
    volumes:
      - mongo_data:/data/db
    ports:
      - "27017:27017"
    networks: [microservices-net]

  # — Application Services —
  product-service:
```

```

build: ./product-service
container_name: product_service
env_file: ./product-service/.env
environment:
  DATABASE_URL: postgresql://admin:secret@postgres:5432/products_db
ports:
  - "3001:3001"
depends_on:
  postgres: { condition: service_healthy }
networks: [microservices-net]

order-service:
build: ./order-service
container_name: order_service
env_file: ./order-service/.env
environment:
  MONGODB_URI: mongodb://admin:secret@mongo:27017/orders_db?authSource=admin
ports:
  - "3002:3002"
depends_on: [mongo]
networks: [microservices-net]

# — API Gateway —————
api-gateway:
build: ./api-gateway
container_name: api_gateway
ports:
  - "3000:3000"
environment:
  PRODUCT_SERVICE_URL: http://product-service:3001
  ORDER_SERVICE_URL: http://order-service:3002
depends_on: [product-service, order-service]
networks: [microservices-net]

```

Các lệnh Docker Compose thường dùng:

```

# Khởi động toàn bộ hệ thống
docker-compose up -d

# Xem logs của một service
docker-compose logs -f product-service

# Chạy Prisma migration trong container
docker-compose exec product-service npx prisma migrate deploy

# Dừng và xoá containers (giữ volumes)
docker-compose down

# Dừng và xoá cả volumes (reset DB)
docker-compose down -v

# Rebuild một service cụ thể
docker-compose up -d --build product-service

```

VIII. DEPLOY LÊN FREE HOST

8.1 Tổng quan các nền tảng Free

Nền tảng	Loại	Free tier	Phù hợp cho
Railway	Backend	\$5 credit/tháng	Node.js services, Docker
Render	Backend	750 gio/thang	Web services, sleep sau 15 phut
Supabase	PostgreSQL	500MB, 2 projects	Hosted PostgreSQL + Auth
MongoDB Atlas	MongoDB	512MB M0 cluster	Hosted MongoDB miễn phí
Vercel	Serverless	Unlimited	API routes, Next.js (không phải Express thuần)

8.2 Deploy lên Railway (Khuyến nghị)

Railway là lựa chọn tốt nhất cho Microservices vì hỗ trợ Docker, có free tier đủ dùng, và giao diện đơn giản.

Bước 1: Chuẩn bị code

1. Tạo file `Dockerfile` cho mỗi service (xem Section VII).
2. Đảm bảo `package.json` có script `"start": "node src/index.js"`
3. Thêm `.dockerignore`:

```
node_modules
.env
*.log
.git
```

Bước 2: Deploy Product Service lên Railway

```
# Cài Railway CLI
npm install -g @railway/cli

# Đăng nhập
railway login

# Tạo project mới
railway init

# Deploy từ thư mục product-service
cd product-service
railway up
```

4. Vào `dashboard.railway.app` → chọn project vừa tạo.
5. Vào tab **Variables** → thêm biến `DATABASE_URL` (lấy từ Supabase).
6. Railway tự detect Dockerfile và build, deploy tự động.
7. Vào tab **Settings** → **Networking** → bật **Public Networking** để có domain public.

Bước 3: Chạy Prisma Migration sau deploy

```
# Chạy migration trực tiếp trên Railway
railway run npx prisma migrate deploy

# Hoặc thêm vào Dockerfile CMD:
CMD ["sh", "-c", "npx prisma migrate deploy && node src/index.js"]
```

8.3 Deploy lên Render (Thay thế)

8. Đẩy code lên GitHub repository.
9. Vào render.com → New → Web Service.
10. Kết nối GitHub repo → chọn branch main.
11. Cấu hình: Environment = Node, Build Command = `npm install && npx prisma generate`, Start Command = `npm start`
12. Thêm Environment Variables: `DATABASE_URL`, `PORT`, `NODE_ENV=production`
13. Click "Create Web Service" và chờ deploy hoàn tất.

⚠ **Quan trọng** Render free tier sẽ tự sleep sau 15 phút không có traffic. Request đầu tiên sẽ chậm ~30s để wake up. Dùng Railway để tránh vấn đề này.

8.4 Cấu hình Environment Variables cho Production

```
# .env.production - KHÔNG commit file này
NODE_ENV=production
PORT=3001

# Supabase PostgreSQL (Connection Pooling mode cho production)
DATABASE_URL=postgresql://postgres.[PROJECT]:[PWD]@aws-0-ap-southeast-1.pooler.supabase.com:6543/postgres?pgbouncer=true

# MongoDB Atlas
MONGODB_URI=mongodb+srv://user:password@cluster0.xxx.mongodb.net/ordersDB?retryWrites=true&w=majority

# JWT Secret - dùng random string dài ≥ 32 ký tự
JWT_SECRET=your-super-secret-random-string-at-least-32-chars

# CORS - cho phép frontend domain
ALLOWED_ORIGINS=https://your-frontend.vercel.app
```

IX. API GATEWAY

API Gateway là điểm vào duy nhất của hệ thống. Client chỉ gọi đến Gateway, Gateway sẽ route request đến đúng service.

9.1 Cài đặt

```
mkdir api-gateway && cd api-gateway
npm init -y
npm install express http-proxy-middleware express-rate-limit cors helmet dotenv
```

9.2 Cấu hình Proxy Routes

```
// src/index.js
const express = require("express");
const { createProxyMiddleware } = require("http-proxy-middleware");
const rateLimit = require("express-rate-limit");
const cors = require("cors");
const helmet = require("helmet");
require("dotenv").config();

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.ALLOWED_ORIGINS?.split(",") || "*" }));

// Rate limiting: tối đa 100 request / 15 phút / IP
const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 100 });
app.use(limiter);

// Health check
app.get("/health", (req, res) => res.json({ status: "ok", gateway: true }));

// Route: /products/* → Product Service
app.use("/api/products", createProxyMiddleware({
  target: process.env.PRODUCT_SERVICE_URL,
  changeOrigin: true,
  on: { error: (err, req, res) => res.status(503).json({ message: "Product Service không khả dụng" }) }
}));

// Route: /orders/* → Order Service
app.use("/api/orders", createProxyMiddleware({
  target: process.env.ORDER_SERVICE_URL,
  changeOrigin: true,
  on: { error: (err, req, res) => res.status(503).json({ message: "Order Service không khả dụng" }) }
}));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`🟡 API Gateway running on port ${PORT}`));
```

X. KIỂM THỬ API

10.1 Kiểm thử qua Swagger UI

14. Khởi chạy product-service: `npm run dev`
15. Mở trình duyệt: `http://localhost:3001/api-docs`
16. Expand endpoint cần test → Click "Try it out" → Điền tham số → Execute.
17. Xem kết quả Response Body và Response Code trực tiếp.

10.2 Bảng test cases cơ bản

Method + URL	Body / Params	Expected	Kiểm tra
GET /api/products	?page=1&limit=5	200	Có pagination, data là array
POST /api/products	{"name":"Test","price":100}	201	Trả về product có id và slug
POST /api/products	{"name":"","price":-1}	422	Trả về errors array chi tiết
GET /api/products/999	—	404	message: "Không tìm thấy"
GET /api/products	?search=iphone&minPrice=20000000	200	Filter hoạt động đúng
DELETE /api/products/1	—	200	isActive = false (soft delete)

10.3 Postman Collection

Tạo Postman Collection để test có tổ chức:

18. Mở Postman → New Collection → đặt tên "Lab2 Microservices".
19. Thêm biến Collection: `base_url = http://localhost:3001`, `gateway_url = http://localhost:3000`
20. Tạo thư mục con: Products, Orders, Auth.
21. Export collection dạng JSON và commit vào Git:
`/postman/Lab2.postman_collection.json`

XI. BÀI TẬP TỰ LÀM

Yêu cầu 1 — Auth Service với JWT (★★★)

Xây dựng Auth Service riêng biệt với các endpoint:

- POST /api/auth/register — Đăng ký tài khoản, hash password bằng bcrypt.
- POST /api/auth/login — Đăng nhập, trả về accessToken (JWT, hết hạn 15 phút) và refreshToken (7 ngày).
- POST /api/auth/refresh — Cấp lại accessToken từ refreshToken hợp lệ.
- GET /api/auth/me — Xem thông tin user hiện tại (cần Bearer token).

Gợi ý: npm install bcryptjs jsonwebtoken, dùng Prisma với PostgreSQL để lưu user.

Yêu cầu 2 — Tích hợp xác thực vào API Gateway (★★★)

Thêm middleware vào API Gateway để xác thực JWT trước khi route request:

```
// Middleware xác thực trong api-gateway/src/middleware/auth.js
const jwt = require("jsonwebtoken");

const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "Chưa đăng nhập" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: "Token không hợp lệ hoặc đã hết hạn" });
  }
};

// Áp dụng cho các route cần xác thực
app.use("/api/orders", authenticate, createProxyMiddleware({ target:
ORDER_SERVICE_URL }));
```

Yêu cầu 3 — Image Upload với Cloudinary (★★)

- Cài đặt: `npm install multer cloudinary multer-storage-cloudinary`.
- Tạo tài khoản Cloudinary miễn phí (cloudinary.com).
- Thêm endpoint: POST `/api/products/:id/image` để upload ảnh sản phẩm.
- Lưu URL ảnh vào field `imageUrl` của Product.

Yêu cầu 4 — Caching với Redis (★★★)

- Thêm Redis service vào `docker-compose.yml` (image: `redis:7-alpine`).
- Cache kết quả GET `/api/products` trong 5 phút.
- Xoá cache khi có thao tác POST/PUT/DELETE.
- Gợi ý: `npm install ioredis`.

Yêu cầu 5 — Viết Swagger đầy đủ cho Order Service (★★)

- Thêm `swagger-jsdoc` vào `order-service`.
- Document tất cả các endpoint với schemas, parameters, responses.
- Thêm authentication vào Swagger (Bearer token).

PHỤ LỤC — CHECKLIST HOÀN THÀNH

STT	Tiêu chí	Điểm
1	Product Service khởi động, kết nối PostgreSQL thành công	10
2	Prisma schema định nghĩa đúng, migration chạy được	10

STT	Tiêu chí	Điểm
3	CRUD API hoạt động đầy đủ (GET/POST/PUT/DELETE)	15
4	Pagination, filtering, sorting hoạt động đúng	10
5	Validation trả về lỗi chi tiết khi dữ liệu sai	10
6	Swagger UI hiển thị và test được tất cả endpoints	15
7	Order Service kết nối MongoDB, CRUD đơn hàng	10
8	Docker Compose khởi động toàn bộ hệ thống	10
9	Deploy thành công ít nhất 1 service lên free host	10
	TỔNG	100

 "Một hệ thống tốt không chỉ chạy được — nó còn phải dễ hiểu, dễ mở rộng, và dễ debug!"